JIMMA UNIVERSITY

JIMMA INSTITUTE OF TECHNOLOGY

FACULTY OF COMPUTING AND INFORMATICS

DEPARTMENT OF SOFTWARE ENGINEERING

FUNDAMENTAL OF SOFTWARE SECURITY LAB ASSIGNMENT:

NAME : MARARA OBSI

ID NO : RU0647/16

SUBMITTED TO :- Mr.ERIMIAS T.

Lab Assignment: Implementing RSA and AES Algorithms using OpenSSL:

Part 1: RSA Implementation

Step 1: Generate RSA Keys

Use OpenSSL to generate a pair of RSA keys.

1. Generate a private key:

```
Win64 OpenSSL Command Prompt

OpenSSL 3.4.0 22 Oct 2024 (Library: OpenSSL 3.4.0 22 Oct 2024)
built on: Tue Oct 22 23:27:41 2024 UTC
platform: VC-WIN64A
options:  bn(64,64)
compiler: cl  /Z7 /Fdossl_static.pdb /Gs0 /GF /Gy /MD /W3 /wd4090 /nologo /O2 -DL_ENDIAN -DOPENSSL_PIC -D"OPENSSL_BUILDI
NG_OPENSSL" -D"OPENSSL_SYS_WIN32" -D"WIN32_LEAN_AND_MEAN" -D"UNICODE" -D"_UNICODE" -D"_CRT_SECURE_NO_DEPRECATE" -D"_WINS
OCK_DEPRECATED_NO_WARNINGS" -D"NDEBUG" -D_WINSOCK_DEPRECATED_NO_WARNINGS -D_WIN32_WINNT=0x0502
OPENSSLDIR: "C:\Program Files\Common Files\SSL"
ENGINESDIR: "C:\Program Files\OpenSSL\lib\engines-3"
MODULESDIR: "C:\Program Files\OpenSSL\lib\ossl-modules"
Seeding source: os-specific
CPUINFO: OPENSSL_ia32cap=0xffdaf38fffebffff:0x9c67a9

C:\Users\mari>openssl genpkey -algorithm RSA -out private_key.pem -pkeyopt rsa_keygen_bits:2048
....+.....+...+.......+...+.................+....+..+.........+....+..+....+..................++++++++++++++++++++++++
++++++++++++++*..+........++++++++++++++++++++++++++++++++++++++++++*....+..+.+.....+......+.+..+.......+......+...+..+...+
....+.....+...+...+.+.........+..+.........+......+.......+...........................+......+..+..............+...+...
+....+............+..............+....+........+.............+............+..+......+.......+...+.....+.+.....+...+.+...
..+....+..+.........+......+......+...+..........+......+...+..+...+...............+......+......+.+...+..............+.
..+...+.+........+.......+...+......+............+...+..+.......+..+.........+.......+.........+........+.+...........+.
+..+...........................+....+.........+..+....+......+...........+.......+...+..+...+.......+...+..+.+..........
.+.+...+..+....+...+................+.+............++++++
......+...+............+..+...+.......+...+............+..+++++++++++++++++++++++++++++++++++++++*.......+++++++++++++++
++++++++++++++++++++++*....+..+...+..........+...+..............+.+..+....+.........+..+.......+...+..+.+..+....+.....
+...+.......+......+........+.........+......+.......+..............+.+.................+......+.+........+.......+.....
..........+.....+......+.......+............+.....++++++
```

2.Extract the public key:

```
C:\Users\mari>openssl rsa -pubout -in private_key.pem -out public_key.pem
writing RSA key
```

3.Verify the keys:

First when we use :

openssl rsa -in private_key.pem -text –noout:

```
C:\Users\mari>openssl rsa -in private_key.pem -text -noout
Private-Key: (2048 bit, 2 primes)
modulus:
    00:c9:ae:ba:c2:d9:f6:57:41:69:de:8b:e7:86:fd:
    24:82:fb:d2:54:1d:93:46:3e:be:49:47:ba:30:3d:
    94:39:46:a5:f4:2e:3d:72:cb:9a:4d:bd:35:38:6b:
    5b:75:93:ae:a0:6f:40:c4:93:55:ec:57:49:51:60:
    88:4d:e3:a0:cf:f9:74:50:88:c3:f0:7b:c1:dc:82:
    6f:74:ce:9b:cf:3c:01:58:e6:21:5d:b4:db:ed:c5:
    ad:88:87:0c:00:8b:04:66:0b:93:27:ce:da:5c:90:
    7d:cf:bd:df:12:37:d0:cd:33:34:ce:66:be:54:59:
    e0:7d:1a:60:92:3d:20:aa:ea:89:63:3c:85:ce:5d:
    69:b2:72:66:91:bf:18:d3:d3:e4:dd:d4:20:f8:71:
    4a:40:92:18:57:66:ae:3a:73:a2:7c:a4:b8:61:67:
    02:16:de:77:86:5f:e3:ca:d5:a7:24:10:50:e4:f8:
    d6:44:28:92:f9:87:79:fa:33:44:0c:3f:fa:dd:ac:
    8b:32:8f:87:ef:6b:d6:bb:92:7a:4c:b3:84:76:2a:
    3d:12:e6:d2:46:9e:2b:ad:f6:63:58:a3:46:27:15:
    27:cf:0b:7c:ac:42:5e:67:8a:47:a1:d6:60:2c:4d:
    63:a6:f4:ab:52:59:b0:72:2d:56:34:e9:4b:0e:11:
    ce:93
publicExponent: 65537 (0x10001)
privateExponent:
    56:99:60:d7:1e:f9:20:41:55:c0:b9:e9:19:f4:e3:
    73:1c:c7:14:d8:38:2f:3e:ec:bd:11:7f:f8:8b:d0:
    74:01:8a:6b:bd:59:24:4c:19:7f:0c:e1:e7:ce:6d:
    50:a0:af:a8:72:ec:e8:1b:6d:97:14:69:c6:78:01:
    3f:14:a1:2d:fc:2a:7e:14:23:b2:44:a9:9a:d2:25:
    22:8e:3e:a7:ee:d7:93:41:09:8d:af:85:89:e4:ca:
    1d:28:03:b6:cd:fa:7f:08:b4:50:b1:08:e1:1b:27:
    f2:86:50:5d:05:bb:72:08:ed:41:3d:2b:b7:93:7a:
    09:ad:c4:4e:b6:d1:aa:a1:53:56:96:92:86:31:41:
    e4:22:55:c1:72:af:35:8a:da:e7:53:73:3f:4d:50:
    c2:29:a4:6b:46:d7:60:bd:70:ee:ab:e0:3e:55:93:
    b0:c9:41:5b:95:b9:8a:3b:ac:75:9e:fe:2c:09:9e:
    d8:19:50:b1:4a:27:d6:4a:96:79:cc:72:2c:41:1d:
    2f:82:af:3b:59:77:e6:64:2d:b0:43:9b:6f:e1:5c:
    c7:f1:40:b9:0c:0b:98:68:cf:39:74:91:24:24:47:
    13:55:88:24:00:d1:d9:01:27:0c:f7:65:47:59:88:
    9f:9b:b4:35:4e:85:af:4d:4f:d3:1b:d4:cc:f9:c3:
    bd
prime1:
    00:fc:3c:0c:70:fc:7b:f3:b6:38:6c:7e:64:12:b5:
    2b:40:86:ea:8b:89:53:91:4c:62:d3:e2:33:9d:7e:
    27:52:53:87:b4:d3:d3:67:82:b0:8b:8f:09:43:1f:
    72:da:38:e2:02:bd:c5:b4:44:50:9a:cc:c0:33:df:
    3e:61:0f:ab:fa:7b:8a:e9:b9:15:88:c7:9d:d4:d0:
    03:6f:ae:dd:c3:ee:d2:98:08:c2:73:71:b3:cc:9a:
    12:c6:bf:5e:9f:b4:7d:72:f4:ae:ad:f6:06:b9:2d:
    c3:28:e4:80:6b:41:18:92:32:23:5f:17:8f:35:5b:
    42:45:bc:3b:dd:6f:3d:c2:75
```

```
    42:45:bc:3b:dd:6f:3d:c2:75
prime2:
    00:cc:b1:7d:2b:88:7d:af:bd:68:7f:16:7e:a1:02:
    e5:e6:03:e1:05:c8:43:7d:2e:4b:41:15:5f:24:fd:
    5f:40:7f:ac:ce:1d:36:cb:3b:60:a2:00:b5:2f:fa:
    6a:7e:b5:11:0a:9c:51:af:d6:26:51:05:97:f4:1c:
    d5:f9:f9:ef:a4:7a:19:93:62:e8:55:a4:70:6f:96:
    99:65:dd:da:98:31:17:f2:f8:fe:57:b4:b3:29:e7:
    a8:ca:e0:db:22:a5:36:fa:79:0f:f6:ac:f1:f2:1b:
    99:0a:ce:78:89:9b:1a:41:2c:5d:4a:b5:d7:9c:da:
    fc:60:0b:9f:8d:1d:e1:1b:e7
exponent1:
    1d:89:fe:bf:00:a2:97:81:00:8c:9c:24:a6:18:37:
    6f:80:1d:59:a8:04:74:60:a8:b6:3d:a8:63:ca:51:
    5a:8f:d3:b5:b6:94:ec:6b:5e:b7:89:51:c1:5d:9e:
    7a:b5:ec:a9:fb:85:7a:be:01:5e:8f:99:f6:b9:2f:
    39:f8:b6:e0:47:b5:95:f7:89:f1:11:c7:87:13:b1:
    43:54:0c:3e:94:ea:6e:c9:9f:d4:1b:3d:be:39:50:
    f7:9b:93:be:4a:49:e2:bb:8a:f5:6e:79:8d:18:31:
    e1:c9:18:cc:f6:bb:a4:eb:60:3f:13:a8:3e:cc:68:
    2f:78:82:de:44:d7:cb:bd
exponent2:
    00:84:2d:d8:f6:53:9d:55:ee:a9:9b:e7:9a:bd:be:
    71:56:a7:0a:98:01:4a:13:7b:e1:7c:95:a0:6a:f2:
    72:9f:e3:d2:63:06:da:6b:42:5c:94:40:80:c0:0a:
    a4:da:4d:72:43:b7:cd:c8:0e:31:90:00:5c:af:76:
    2f:76:cc:be:c3:27:55:7c:a7:52:00:7b:bf:5f:4f:
    90:9f:07:34:31:33:02:94:13:d5:aa:2d:35:c4:c9:
    a3:65:de:ed:a2:e1:ad:2e:f1:1e:2d:dc:60:20:47:
    51:fc:78:77:bd:e4:3a:7f:fb:76:02:d8:89:f2:f0:
    36:28:8f:58:e8:77:72:f5:e1
coefficient:
    19:f7:6a:f2:12:1a:77:cf:06:b9:c6:b7:5b:d8:60:
    9f:b2:e5:69:fa:2c:7e:23:21:cd:cc:e3:24:03:7b:
    7f:23:14:0f:0d:bc:b4:6f:56:d0:0a:3e:e6:9c:bd:
    a5:67:50:f9:5f:2f:d9:52:77:06:f1:ee:5c:0e:ad:
    ef:f4:a2:3e:ae:3f:ad:71:ba:cf:f8:b2:50:89:e9:
    9c:4e:08:7c:20:62:5e:80:2e:43:13:e1:e2:68:74:
    11:2e:f5:d0:75:82:f8:f6:e9:ad:8b:b7:5d:b9:cf:
    aa:73:58:c8:45:27:f1:8a:d4:19:dc:02:b0:55:a0:
    ff:90:0c:e4:e0:85:d4:46
```

With this command openssl rsa -pubin -in public_key.pem -text –noout:

```
C:\Users\mari>openssl rsa -pubin -in public_key.pem -text -noout
Public-Key: (2048 bit)
Modulus:
    00:c9:ae:ba:c2:d9:f6:57:41:69:de:8b:e7:86:fd:
    24:82:fb:d2:54:1d:93:46:3e:be:49:47:ba:30:3d:
    94:39:46:a5:f4:2e:3d:72:cb:9a:4d:bd:35:38:6b:
    5b:75:93:ae:a0:6f:40:c4:93:55:ec:57:49:51:60:
    88:4d:e3:a0:cf:f9:74:50:88:c3:f0:7b:c1:dc:82:
    6f:74:ce:9b:cf:3c:01:58:e6:21:5d:b4:db:ed:c5:
    ad:88:87:0c:00:8b:04:66:0b:93:27:ce:da:5c:90:
    7d:cf:bd:df:12:37:d0:cd:33:34:ce:66:be:54:59:
    e0:7d:1a:60:92:3d:20:aa:ea:89:63:3c:85:ce:5d:
    69:b2:72:66:91:bf:18:d3:d3:e4:dd:d4:20:f8:71:
    4a:40:92:18:57:66:ae:3a:73:a2:7c:a4:b8:61:67:
    02:16:de:77:86:5f:e3:ca:d5:a7:24:10:50:e4:f8:
    d6:44:28:92:f9:87:79:fa:33:44:0c:3f:fa:dd:ac:
    8b:32:8f:87:ef:6b:d6:bb:92:7a:4c:b3:84:76:2a:
    3d:12:e6:d2:46:9e:2b:ad:f6:63:58:a3:46:27:15:
    27:cf:0b:7c:ac:42:5e:67:8a:47:a1:d6:60:2c:4d:
    63:a6:f4:ab:52:59:b0:72:2d:56:34:e9:4b:0e:11:
    ce:93
Exponent: 65537 (0x10001)
```

Step 2: Encrypt and Decrypt a Message

1. Create a plaintext message file (e.g., message.txt).
   echo " Write your name and ID here" > message.txt
2. Encrypt the message using the public key:
   openssl rsautl -encrypt -inkey public_key.pem -pubin -in message.txt -out encrypted_message.bin
3. Decrypt the message using the private key:
   openssl rsautl -decrypt -inkey private_key.pem -in encrypted_message.bin -out decrypted_message.txt
4. Verify the decrypted message:
   cat decrypted_message.txt

```
C:\Users\mari>echo Name:Marara Obsi IDNO:RU0647/16 > message.txt

C:\Users\mari>openssl pkeyutl -encrypt -inkey public_key.pem -pubin -in message.txt -out encrypted_message.bin

C:\Users\mari>openssl pkeyutl -decrypt -inkey private_key.pem -in encrypted_message.bin -out decrypted_message.txt

C:\Users\mari>cat aes_decrypted_message.txt
'cat' is not recognized as an internal or external command,
operable program or batch file.

C:\Users\mari>type decrypted_message.txt
Name:Marara Obsi IDNO:RU0647/16
```

Part 2: AES Implementation:

Step 1: Generate an AES Key

1. Generate a 256-bit AES key:
   openssl rand -base64 32 > aes_key.bin

Step 2: Encrypt and Decrypt a Message

1. Encrypt a message file using AES:
   openssl enc -aes-256-ecb -salt -in message.txt -out aes_encrypted_message.bin -pass file:./aes_key.bin
2. Decrypt the AES-encrypted message:
   openssl enc -d -aes-256-ecb -in aes_encrypted_message.bin -out aes_decrypted_message.txt -pass file:./aes_key.bin
3. Verify the decrypted message:
   cat aes_decrypted_message.txt

```
C:\Users\mari>openssl rand -base64 32 > aes_key.bin

C:\Users\mari>openssl enc -aes-256-ecb -salt -in message.txt -out aes_encrypted_message.bin -pass file:./aes_key.bin
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.

C:\Users\mari>openssl enc -d -aes-256-ecb -in aes_encrypted_message.bin -out aes_decrypted_message.txt -pass file:./aes_key.bin
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.

C:\Users\mari>type aes_decrypted_message.txt
Name:Marara Obsi IDNO:RU0647/16
```

Part 3: Hybrid Encryption Scheme

In this section, you will use RSA to securely exchange the AES key and IV, and then use AES to encrypt the message.

Step 1: Encrypt the AES Key with RSA

1. Encrypt the AES key:
   openssl rsautl -encrypt -inkey public_key.pem -pubin -in aes_key.bin -out encrypted_aes_key.bin

Step 2: Decrypt the AES Key and IV with RSA

1. Decrypt the AES key:
   openssl rsautl -decrypt -inkey private_key.pem -in encrypted_aes_key.bin -out decrypted_aes_key.bin

Step 3: Encrypt and Decrypt a Message Using the Hybrid Scheme

1. Encrypt the message using the AES key:
   openssl enc -aes-256-ecb -salt -in message.txt -out hybrid_encrypted_message.bin -pass file:./decrypted_aes_key.bin
2. Decrypt the hybrid-encrypted message:
   openssl enc -d -aes-256-ecb -in hybrid_encrypted_message.bin -out hybrid_decrypted_message.txt -pass file:./decrypted_aes_key.bin
3. Verify the decrypted message:
   cat hybrid_decrypted_message.txt

```
C:\Users\mari>openssl rsautl -encrypt -inkey public_key.pem -pubin -in aes_key.bin -out encrypted_aes_key.bin
The command rsautl was deprecated in version 3.0. Use 'pkeyutl' instead.

C:\Users\mari>openssl pkeyutl -encrypt -inkey public_key.pem -pubin -in aes_key.bin -out encrypted_aes_key.bin

C:\Users\mari>openssl pkeyutl -decrypt -inkey private_key.pem -in encrypted_aes_key.bin -out decrypted_aes_key.bin

C:\Users\mari>openssl enc -d -aes-256-ecb -in hybrid_encrypted_message.bin -out hybrid_decrypted_message.txt -pass file:./decrypted_a
es_key.bin
Can't open "hybrid_encrypted_message.bin" for reading, No such file or directory
785A0000:error:80000002:system library:BIO_new_file:No such file or directory:crypto\bio\bss_file.c:67:calling fopen(hybrid_encrypted
_message.bin, rb)
785A0000:error:10000080:BIO routines:BIO_new_file:no such file:crypto\bio\bss_file.c:75:

C:\Users\mari>openssl enc -aes-256-ecb -salt -in message.txt -out hybrid_encrypted_message.bin -pass file:./decrypted_aes_key.bin
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.

C:\Users\mari>openssl enc -d -aes-256-ecb -in hybrid_encrypted_message.bin -out hybrid_decrypted_message.txt -pass file:./decrypted_a
es_key.bin
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.

C:\Users\mari>type hybrid_decrypted_message.txt
Name:Marara Obsi IDNO:RU0647/16
```

A description of any challenges faced and how you resolved them.

1) Command Compatibility Issues in Windows:

   Challenge:

   Some commands from the lab guide (such as echo, cat, and rsautl) were written for Linux-based terminals. When running them in Windows Command Prompt, errors occurred such as:

   - 'cat' is not recognized…
   - '$echo' is not recognized…
   - rsautl was deprecated in version 3.0

   These issues happened because Windows uses different command syntax, and OpenSSL 3.x has replaced rsautl with pkeyutl.

   Resolution:

The commands were adapted to Windows equivalents:

- cat → replaced with type
- echo → used without $
- rsautl → replaced with pkeyutl for encryption and decryption

2) Understanding the Difference Between RSA and AES Usage

Challenge:

At first, it was confusing why RSA was not used directly to encrypt the whole message, and why AES was needed in addition to RSA.

Resolution:

After reviewing cryptography documentation, it became clear that

: ● RSA is slow and is mainly used for key exchange

- AES is fast and is used to encrypt the actual data

Therefore, the lab follows a hybrid encryption model:

- RSA encrypts the AES key
- AES encrypts the message

This approach reflects real-world systems such as TLS.